Trabalho Prático - Estrutura de Dados II

Tema: Implementação de uma Árvore B Dinâmica com Acesso a Arquivos Binários

Descrição: Neste trabalho, você deverá implementar uma árvore B em linguagem C, onde cada nó será armazenado em um arquivo binário. A árvore deve suportar as operações de busca, inserção e remoção, conforme descrito em sala e apresentado nos slides (CLRS). Além disso, a árvore B deverá ser dinâmica, ou seja, o grau mínimo *t* deve ser configurável e determinado pelo usuário durante a execução do programa (alocação dinâmica). Também é necessário implementar uma função para imprimir a árvore B na tela de maneira visualmente compreensível.

Requisitos:

1. **Implementação das Operações:**
    - **Busca:** Implemente a operação de busca na árvore B. A busca deve ser realizada nos arquivos binários que representam os nós da árvore.
    - **Inserção (CLRS):** Implemente a inserção de novos elementos na árvore B, conforme o algoritmo de inserção CLRS. A inserção deve manter a propriedade de balanceamento da árvore B (quantidade de nós entre *t-1* e *2t-1*).
    - **Remoção (CLRS):** Implemente a remoção de elementos da árvore B, seguindo o algoritmo de remoção CLRS. A remoção deve garantir que a árvore permaneça balanceada após a operação (quantidade de nós entre *t-1* e *2t-1*)..
2. **Árvore B Dinâmica:**
    - O grau mínimo *t* da árvore B deverá ser configurável pelo usuário durante a execução do programa.
    - A implementação deve permitir que a árvore B funcione corretamente com qualquer valor de *t* especificado, respeitando as propriedades da árvore B.
3. **Armazenamento em Arquivos Binários:**
    - Cada nó da árvore B deve ser armazenado em um arquivo binário separado.
    - O nome dos arquivos deve ser gerado dinamicamente, garantindo que cada nó seja acessado corretamente durante as operações de busca, inserção e remoção.
4. **Impressão da Árvore B:**
    - Implemente uma função que imprima a árvore B na tela, de forma que a hierarquia dos nós seja clara.
    - A impressão deve mostrar os nós em seus níveis corretos, facilitando a compreensão visual da estrutura da árvore.
        i. Somente será possível imprimir corretamente arvores com *t* “pequenos”.

Exemplo de Impressão da Árvore B:

```
Nível 1:         | 18 |
                 /    \
Nível 2:     | 10 |    | 25 |
             /    \    /    \
Nível 3: | 6 | | 12 | | 20 | | 30 |
```

Exemplo de `struct` para um Nó da Árvore B:

```
typedef struct BTreeNode {
    int n;                  // Número de chaves armazenadas no nó
    int *keys;              // Chaves armazenadas no nó
    char **children;        // Nomes dos arquivos filhos
    bool leaf;              // Indica se o nó é uma folha
} BTreeNode;

// Função para criar um nó da árvore B
BTreeNode* createNode(int T, bool leaf) {
    BTreeNode* node = (BTreeNode*) malloc(sizeof(BTreeNode));
    node->n = 0;
    node->keys = (int*) malloc((2*T - 1) * sizeof(int));
    node->children = (char**) malloc((2*T) * sizeof(char*));
    node->leaf = leaf;
    return node;
}
```

Entregáveis:

1. Código-fonte completo em C, documentado.
2. Relatório explicando as decisões de implementação, a dinâmica do grau *t*, e os testes realizados para verificar a corretude da árvore B.

Critérios de Avaliação: A avaliação considerará a corretude da implementação, a eficiência das operações de busca, inserção e remoção, a flexibilidade da implementação para diferentes valores de *t*, e a clareza da impressão da árvore na tela. O código deve ser bem documentado e seguir boas práticas de programação.